Reviewer Pack — Minimal Index of Tropical Curves over Riemann Surfaces and R...

Entry 01fd875ec752 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 00:08:36 UTC

Minimal Index of Tropical Curves over Riemann Surfaces and Read-Twice BP Size

Entry ID: `01fd875ec752` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 00:08:36 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry × Algebraic Geometry: Riemann Surfaces **Field B** (complexity object): Branching Program: Read-Twice Complexity

Statement:

> ['For every read-twice BP P with n variables, there exists a Riemann surface over the complex numbers with at most O(n) points, such that the index of the tropical curve associated with the Riemann surface is bounded by the size of P, i.e., index(tropical_curve) ≤ 2^size(P).', 'For any read-twice BP P with n variables and size(P) = Θ(2^n), there exists a Riemann surface over the complex numbers with at most O(n) points such that its tropical curve has an index of at least Ω(size(P)).']

Rationale (proposer's reasoning):

> ['The study of Riemann surfaces can provide insights into the geometric properties of algebraic varieties. Tropical geometry allows for a combinatorial representation of these surfaces, which may reveal new relationships between geometric invariants and computational problems.', 'Previous work has shown connections between tropical curves and complexity theory, particularly in understanding the size of read-twice BPs. This conjecture proposes an explicit link between Riemann surface indices and BP sizes, suggesting that geometric properties might provide a stronger invariant for BP size.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b7196c68c5b1bce0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For read-twice BPs with n variables and size(P) = Θ(2^n), if the tropical curve's index is at least half of P's size, or for sizes ≤ Θ(2^n), if the index is less than or equal to P's size.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"tropical geometry" AND "algebraic geometry" AND Riemann surfaces` - `"read-twice BP" AND "index of tropical curves" AND "Riemann surface"` - `"complex numbers" AND "size(P) = θ(2^n)" AND "tropical curve index"`

Top relevant hits considered: - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/2307.15946v1] Gamma conjecture and tropical geometry - [http://arxiv.org/abs/1706.05401v3] Counting curves on Hirzebruch surfaces: tropical geometry and the Fock space - [s2:508317fb7c22eb34ca056ad95c4f2336f3c75bfb] Repeated Patterns in Arbitrary Colorings in Projective Over Finite Numbers of Real Cyclotomic Fields, Principal Ideals,

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def read_twice_bp(n):
    if n == 1:
        return [0]
    bp = []
    for i in range(2 ** (n - 1) - 1):
        bp.append(bp[i // 2] ^ (i % 2))
    return bp

def calculate_tropical_curve_index(bp, n):
    # Simplified tropical curve index calculation
    return len(bp)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        size_P = 2 ** n
        bp = read_twice_bp(n)
```

```
        if len(bp) != size_P - 1:
            return {
                "metric_name": "tropical_curve_index",
                "metric_value": None,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": "bp_size_mismatch"
            }

        index = calculate_tropical_curve_index(bp, n)
        results.append(index)

    mean_index = sum(results) / len(results)
    max_index = max(results)

    if all(index <= size_P for index in results):
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = f"index={max_index} > size(P)={size_P}"

    return {
        "metric_name": "tropical_curve_index",
        "metric_value": mean_index,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    if not sys.argv[1:]:
        seeds = [2**i + 1 for i in range(5, 6)]  # Default list of 30 primes
    else:
        seeds = [int(seed) for seed in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean_value = sum(results) / len(results)
    support_fraction = sum(1 for r in results if r is not None and r <= 2**math.ceil(math.log2(len

    if all(r is not None and r <= 2**math.ceil(math.log2(len(results))) for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=NA support_fraction={support_fraction}")
    elif any(r is not None and r > 2**math.ceil(math.log2(len(results))) for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result is not No
        print(f"RESULT: FALSIFIED counterexample='index_exceeds_size_P' first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE reason=metric_value_none_or_unsupported")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34ea2315.py", line 63, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34ea2315.py", line 38, in run_trial
    bp = read_twice_bp(n)
         ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34ea2315.py", line 25, in read_twice_bp
    bp = [0] * (2 ** n - 1)
         ~~~~^~~~~~~~~~~~~~
MemoryError
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed due to a MemoryError before producing data, which means the pre-registered support condition was not evaluated. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15517
2	preregistration	ollama_remote	glm4:latest	0	0	9110
3	novelty	ollama_remote	glm4:latest	0	0	10609
4	novelty	ollama_remote	glm4:latest	0	0	9606
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12007
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9280
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6640
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8294
9	judge	ollama_remote	glm4:latest	0	0	14632

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 95696 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/01fd875ec752.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/01fd875ec752.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/01fd875ec752.tar.gz` (if generated)